Coffee Shop Application on AWS EKS

End-to-End Project Documentation

This document records the final implementation, Kubernetes manifests, application components, Docker images, deployment process, database persistence, troubleshooting, and successful validation of the Coffee Shop application.

1. Final Architecture

The final application flow is:

```
User Browser
    |
    v
AWS EC2 Public IP :30080
    |
    v
Kubernetes NodePort
    |
    v
Frontend NGINX
    |
    | /api/*
    v
coffee-backend Kubernetes Service :8080
    |
    v
Spring Boot Backend
    |
    v
MySQL Service :3306
    |
    v
MySQL Pod
    |
    v
PVC mysql-pvc
    |
    v
AWS EBS volume
```

2. Final Container Images

Component	Final / Intended Image	Purpose
Frontend	raveendradudyala/coffee-shop-frontend:3.0.1	HTML/CSS/JavaScript + NGINX
Backend	raveendradudyala/coffee-shop-backend:5.0.0	Spring Boot REST API
Database	mysql:8.0	MySQL database

Note: 3.0.1 was built successfully. The documented deployment target is 3.0.1; the final EKS image should be verified with kubectl after the push/set-image step.

3. Project Structure

```
/eks/coffee-shop-frontend/
|
+-- coffee-shop-frontend/
|   +-- index.html
|   +-- style.css
|   +-- app.js
|   +-- Dockerfile
|   +-- nginx.conf
|
+-- coffee-shop-backend/
    +-- pom.xml
    +-- Dockerfile
    +-- src/main/java/com/coffeeshop/
    |   +-- OrderController.java
```

```
    |   +-- Order.java
    +-- src/main/resources/
        +-- application.properties

Kubernetes manifests:
    /eks/coffee-shop-frontend/manifest/
    +-- 03-mysql.yaml
    +-- backend Deployment/Service manifest
    +-- frontend Deployment/Service manifest
    +-- EBS StorageClass manifest
```

4. Frontend

The frontend is a static Coffee Shop application served by NGINX. It contains the menu, cart, checkout flow, and Order History UI.

The Order History feature calls GET /api/orders and displays order ID, total amount, date/time, and saved status. The final frontend version created for the click/UI fix is 3.0.1.

5. NGINX Configuration

```
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://coffee-backend:8080/api/;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Important correction: the original NGINX configuration used backend:8080. The actual Kubernetes Service is coffee-backend, so proxy_pass was changed to coffee-backend:8080.

6. Backend Application

The backend is a Spring Boot 3.5.5 application running on Java 17 and port 8080.

Dependencies include Spring Web, Spring JDBC, and MySQL Connector/J.

Backend Dockerfile:

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/coffee-shop-backend-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

7. Backend Database Configuration

```
server.port=8080

spring.datasource.url=jdbc:mysql://${MYSQL_SERVICE_HOST}:${MYSQL_SERVICE_PORT}/coffeeshop
spring.datasource.username=root
spring.datasource.password=${MYSQL_ROOT_PASSWORD}

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.datasource.hikari.maximum-pool-size=5
spring.datasource.hikari.minimum-idle=1

spring.jpa.hibernate.ddl-auto=none
```

Kubernetes automatically provides MYSQL_SERVICE_HOST and MYSQL_SERVICE_PORT environment variables for the MySQL Service. MYSQL_ROOT_PASSWORD is supplied from the Kubernetes Secret.

8. REST APIs

Method	Endpoint	Function
POST	/api/orders	Creates an order and saves order_id and total_amount
GET	/api/orders	Returns saved orders from MySQL

9. OrderController

```
POST /api/orders
    |
    +-- receives product list
    +-- calculates total
    +-- generates order ID
    +-- INSERT INTO orders
    +-- returns created order

GET /api/orders
    |
    +-- SELECT order_id, total_amount, created_at
    +-- ORDER BY created_at DESC
    +-- returns JSON
```

The GET API was successfully tested from inside Kubernetes and returned orders 1002 and 1001.

10. MySQL Manifest

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
type: Opaque
stringData:
  MYSQL_ROOT_PASSWORD: dbpassword11

---
apiVersion: v1
kind: PersistentVolumeClaim
```

```
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: ebs-sc

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_ROOT_PASSWORD
          volumeMounts:
            - name: mysql-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-storage
          persistentVolumeClaim:
            claimName: mysql-pvc

---
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  type: ClusterIP
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
```

11. Storage Design

```
MySQL
  |
  v
mysql-pvc (5Gi, ReadWriteOnce)
  |
  v
StorageClass: ebs-sc
  |
  v
AWS EBS volume
```

This means database data is stored on persistent EBS-backed storage rather than only inside the MySQL container filesystem.

12. Complete Deployment Process

1. Create the application source structure.
2. Build the Spring Boot backend with Maven.
3. Create the backend Docker image.
4. Push the backend image to Docker Hub.
5. Create EBS StorageClass and MySQL PVC.
6. Deploy MySQL and expose it internally using the mysql ClusterIP Service.
7. Configure Spring Boot to connect to MySQL using Kubernetes Service environment variables and Secret.
8. Deploy the backend and expose it internally using the coffee-backend ClusterIP Service.
9. Build the frontend NGINX image.
10. Configure NGINX to proxy /api/ to coffee-backend:8080.
11. Expose the frontend using a Kubernetes NodePort.
12. Test POST /api/orders.
13. Test GET /api/orders from inside Kubernetes.
14. Implement Order History in the frontend.
15. Clean duplicate Order History JavaScript/CSS created during iteration.
16. Validate JavaScript using node --check.
17. Build frontend image 3.0.1.
18. Push frontend image to Docker Hub.
19. Update the EKS frontend Deployment and wait for rollout.

13. Important Commands Used

```
# Verify EKS workloads
kubectl get pods
kubectl get svc

# Test backend from inside Kubernetes
kubectl run curl-test \
  --image=curlimages/curl \
  --rm -it \
  --restart=Never \
  -- curl http://coffee-backend:8080/api/orders

# Test external API through frontend NGINX
curl -i http://<EC2-PUBLIC-IP>:30080/api/orders

# Frontend syntax validation
node --check app.js

# Build frontend
docker build -t raveendradudyala/coffee-shop-frontend:3.0.1 .

# Push frontend
docker push raveendradudyala/coffee-shop-frontend:3.0.1

# Deploy frontend
kubectl set image deployment/coffee-frontend \
coffee-frontend=raveendradudyala/coffee-shop-frontend:3.0.1

# Wait for rollout
kubectl rollout status deployment/coffee-frontend
```

14. Successful Validation

The backend API was successfully tested from a temporary Kubernetes curl pod and returned:


```
[
  {
    "order_id": 1002,
    "total_amount": 320.0,
    "created_at": "2026-09-22T19:28:43.000+00:00"
  },
  {
    "order_id": 1001,
    "total_amount": 280.0,
    "created_at": "2026-09-22T19:25:09.000+00:00"
  }
]
```


The external request through the frontend NodePort was also verified with HTTP/1.1 200 and the same order data.

Frontend pods were observed in Running/Ready state, and the frontend 3.0.1 Docker build completed successfully.

15. Troubleshooting Performed

Issue	Root cause / fix
coffee-backend could not be resolved from EC2 host	Kubernetes Service DNS names resolve inside Kubernetes workloads; testing was moved to a curl pod.
NGINX used backend:8080	Changed upstream to coffee-backend:8080, matching the Kubernetes Service name.
GET /api/orders initially returned an application error	Backend image was updated to the version containing the GET endpoint.
Order History JavaScript was duplicated	Removed duplicate implementations and kept one clean implementation.
Order History CSS was duplicated	Removed old/appended duplicate blocks and kept one implementation.
Browser button did not initially behave correctly	Reworked dynamic button creation using addEventListener and rebuilt frontend as 3.0.1.

16. Final Application Behavior

```
Customer
   |
   +--> Browse menu
   |
   +--> Add products to cart
   |
   +--> Checkout
           |
           v
       POST /api/orders
           |
           v
        MySQL
           |
           v
       EBS storage

Order History
   |
   +--> GET /api/orders
   |
   v
Saved orders from MySQL
   |
   v
Browser displays:
    Order ID
    Total
    Date/Time
    Saved status
```

17. Current Known Data Model Limitation

The current orders table stores order_id, total_amount, and created_at. The backend POST receives the product list and returns it in the immediate response, but historical item-level details are not currently persisted in a separate order_items table. Therefore Order History currently displays order-level information, not the individual products in each historical order.

18. Production Improvements

• Use a strong generated database password and a proper secret-management solution.

• Replace the in-memory AtomicLong order ID generator with a database-generated AUTO_INCREMENT ID or UUID.

• Persist order items in a separate order_items table if historical product details are required.

• Use a production ingress/load balancer rather than NodePort for internet-facing traffic.

• Add health/readiness probes and resource requests/limits.

• Add centralized logging and monitoring.

• Use CI/CD or GitOps to build, scan, publish, and deploy immutable image versions.

19. Final Summary

The project demonstrates an end-to-end containerized Coffee Shop application deployed on AWS EKS. The frontend is served by NGINX, API traffic is reverse-proxied to a Spring Boot backend, the backend stores orders in MySQL, and MySQL uses a 5Gi EBS-backed PersistentVolumeClaim. The Order History feature retrieves persisted orders through the backend API. The final frontend image built during the last fix is 3.0.1, the backend target is 5.0.0, and MySQL uses mysql:8.0.